Ex. No. 5 Date: 09/09/2026	SUBQUERIES AND CORRELATED SUBQUERIES	URK24CS1021

Aim

To execute the given queries using the concept of subqueries and correlated subqueries.

Description

A subquery is a SELECT statement nested inside another SQL statement, typically in the WHERE clause, used when the value to search for isn't known in advance but can itself be produced by a query. A correlated subquery references a column from the outer query in its own WHERE clause, so it is re-evaluated once for every row the outer query processes -- unlike an ordinary subquery, which runs once and is reused for every outer row. EXISTS and NOT EXISTS test only whether a correlated subquery returns any rows at all.

Questions

1. Retrieve users who have reserved for a sports tournament.

```
SELECT name FROM URK24CS1021_users WHERE id IN
    (SELECT userid FROM URK24CS1021_ticket WHERE eventid IN
        (SELECT eventid FROM URK24CS1021_event WHERE name = 'Sports Tournament'));
```

```
psql — labdb

labdb=# SELECT name FROM URK24CS1021_users WHERE id IN
labdb(#     (SELECT userid FROM URK24CS1021_ticket WHERE eventid IN
labdb(#         (SELECT eventid FROM URK24CS1021_event WHERE name = 'Sports Tournament'));
   name
----------
 Bob Smith
(1 row)

labdb=#
```

2. Find events hosted in London.

```
SELECT name FROM URK24CS1021_event WHERE venueid IN
    (SELECT venueid FROM URK24CS1021_location WHERE city = 'London');
```

```
psql — labdb

labdb=# SELECT name FROM URK24CS1021_event WHERE venueid IN
labdb(#     (SELECT venueid FROM URK24CS1021_location WHERE city = 'London');
       name
------------------
 Sports Tournament
 Charity Concert
(2 rows)

labdb=#
```

3. Calculate average ticket price for an event Art Exhibition.

```
SELECT AVG(price) AS avg_price FROM URK24CS1021_ticket WHERE eventid =
    (SELECT eventid FROM URK24CS1021_event WHERE name = 'Art Exhibition');
```

```
psql — labdb

labdb=# SELECT AVG(price) AS avg_price FROM URK24CS1021_ticket WHERE eventid =
labdb(#     (SELECT eventid FROM URK24CS1021_event WHERE name = 'Art Exhibition');
      avg_price
--------------------
 50.0000000000000000
(1 row)

labdb=#
```

4. Retrieve users who haven't made any reservations.

```
SELECT name FROM URK24CS1021_users u WHERE NOT EXISTS
    (SELECT 1 FROM URK24CS1021_ticket t WHERE t.userid = u.id);
```

```
psql — labdb

labdb=# SELECT name FROM URK24CS1021_users u WHERE NOT EXISTS
labdb(#     (SELECT 1 FROM URK24CS1021_ticket t WHERE t.userid = u.id);
   name
-----------
 David Kim
(1 row)

labdb=#
```

5. Calculate the maximum price among all events reserved by a user.

```
SELECT MAX(price) AS max_price FROM URK24CS1021_ticket WHERE userid =
    (SELECT id FROM URK24CS1021_users WHERE name = 'Alice Johnson');
```

```
psql — labdb

labdb=# SELECT MAX(price) AS max_price FROM URK24CS1021_ticket WHERE userid =
labdb(#     (SELECT id FROM URK24CS1021_users WHERE name = 'Alice Johnson');
 max_price
-----------
    200.00
(1 row)

labdb=#
```

6. Find users who have reserved tickets for events on 20th July 2023.

```
SELECT DISTINCT u.name FROM URK24CS1021_users u WHERE u.id IN
    (SELECT t.userid FROM URK24CS1021_ticket t WHERE t.eventid IN
        (SELECT eventid FROM URK24CS1021_event WHERE eventdate = '2023-07-20'));
```

```
psql — labdb

labdb=# SELECT DISTINCT u.name FROM URK24CS1021_users u WHERE u.id IN
labdb(#     (SELECT t.userid FROM URK24CS1021_ticket t WHERE t.eventid IN
labdb(#         (SELECT eventid FROM URK24CS1021_event WHERE eventdate = '2023-07-20'));
     name
--------------
 Alice Johnson
(1 row)

labdb=#
```

7. Find users who have reserved tickets for events at USA.

```
SELECT DISTINCT u.name FROM URK24CS1021_users u WHERE EXISTS
    (SELECT 1 FROM URK24CS1021_ticket t, URK24CS1021_event e, URK24CS1021_location l
     WHERE t.userid = u.id AND t.eventid = e.eventid
       AND e.venueid = l.venueid AND l.country = 'USA');
```

```
psql — labdb

labdb=# SELECT DISTINCT u.name FROM URK24CS1021_users u WHERE EXISTS
labdb(#     (SELECT 1 FROM URK24CS1021_ticket t, URK24CS1021_event e, URK24CS1021_location l
labdb(#      WHERE t.userid = u.id AND t.eventid = e.eventid
labdb(#        AND e.venueid = l.venueid AND l.country = 'USA');
     name
--------------
 Alice Johnson
 Ethan Brown
(2 rows)

labdb=#
```

8. Retrieve events where the price of the events is less than $.30.

```
SELECT DISTINCT e.name FROM URK24CS1021_event e WHERE e.eventid IN
    (SELECT eventid FROM URK24CS1021_ticket WHERE price < 30);
```

```
psql — labdb

labdb=# SELECT DISTINCT e.name FROM URK24CS1021_event e WHERE e.eventid IN
labdb(#     (SELECT eventid FROM URK24CS1021_ticket WHERE price < 30);
      name
-----------------
 Charity Concert
(1 row)

labdb=#
```

9. Calculate the total revenue generated by Ethan.

```
SELECT SUM(price) AS total_revenue FROM URK24CS1021_ticket WHERE userid =
    (SELECT id FROM URK24CS1021_users WHERE name = 'Ethan Brown');
```

```
psql — labdb

labdb=# SELECT SUM(price) AS total_revenue FROM URK24CS1021_ticket WHERE userid =
labdb(#     (SELECT id FROM URK24CS1021_users WHERE name = 'Ethan Brown');
 total_revenue
---------------
        150.00
(1 row)

labdb=#
```

10. Find the events which have been conducted at the same venue.

```
SELECT name FROM URK24CS1021_event e WHERE EXISTS
    (SELECT 1 FROM URK24CS1021_event e2
     WHERE e2.venueid = e.venueid AND e2.eventid <> e.eventid)
ORDER BY name;
```

```
psql — labdb

labdb=# SELECT name FROM URK24CS1021_event e WHERE EXISTS
labdb(#     (SELECT 1 FROM URK24CS1021_event e2
labdb(#      WHERE e2.venueid = e.venueid AND e2.eventid <> e.eventid)
labdb(# ORDER BY name;
       name
-------------------
 Charity Concert
 Sports Tournament
 Startup Summit
 Tech Conference
(4 rows)

labdb=#
```

Result

The queries were executed and the desired output was obtained.